

VNU-HCM University of Information Technology

ASSIGNMENT 3

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
4-25-2025

Mục lục

Lời mở đầu.....	2
1. Basic	3
1.1. Kiểm tra biểu thức toán học (Latex)	3
1.2. LinkedList-Insertion.....	5
1.3. LinkedList-NhapDaThuc	6
1.4. LinkedList-Reverse	8
1.5. Dec to Bin	10
1.6. LinkedList: MergetwoSortedLinkedList.....	11
1.7. Tree: Preorder Traversal(NLR) - Duyệt cây BST theo NLR	13
1.8. Tree: Preorder Traversal (NLR) II - Duyệt cây BST theo NLR không đệ quy.....	14
1.9. LinkedList: Tìm nút chung của 2 danh sách liên kết đơn	15
1.10. Binary Search Tree: Nút tổ tiên thấp nhất (tiếng Việt)	17
2. Advance	18
2.1. Kiểm Kê 2	18
2.2. Binary Search 2	22
2.3. khanhgd.DetectVirusin2D.....	25
2.4. khanhgd.XepHang2.....	29
2.5. khanhgd.XepHang.....	31

GITHUB: [LINK](#)

Lời mở đầu

Bài tập tuần 3 cũng được chia thành 2 mục **Advance** và **Basic**. Tương ứng với nó là mức độ khó dễ cũng được phân hóa rõ ràng để người làm có thể ôn lại những kiến thức về danh sách liên kết và cây nhị phân. Và sau đây là ý tưởng cũng như cách giải các bài tập được đưa ra:

[IT003.P21.CTTN.2] Assignment 3 (Basic) 

10 problems with a total score of 1000

#	PROBLEM	SCORE
1	Kiểm tra biểu thức toán học (Latex)	100
2	LinkedList-Insertion	100
3	LinkedList-NhapDaThuc	100
4	LinkedList-Reverse	100
5	Dec to Bin	100
6	LinkedList: MergeTwoSortedLinkedList	100
7	Tree: Preorder Traversal (NLR) - Duyệt cây BST theo NLR	100
8	Tree: Preorder Traversal (NLR) II - Duyệt cây BST theo NLR không đệ quy	100
9	LinkedList: Tìm nút chung của 2 danh sách liên kết đơn	100
10	Binary Search Tree: Nút tổ tiên thấp nhất (tiếng Việt)	100

[IT003.P21.CTTN.2] Assignment 3 (Advance) 

5 problems with a total score of 500

#	PROBLEM	SCORE
1	Kiểm Kê 2	100
2	Binary Search 2	100
3	khangtd.DetectVirusin2D	100
4	khangtd.XepHang2	100
5	khangtd.XepHang	100

1. Basic

1.1. Kiểm tra biểu thức toán học (Latex)

➤ Đề bài:

Kiểm tra tính hợp lệ của chuỗi *latex*?

Biểu thức toán học giúp chúng ta diễn đạt tự nhiên thành các phương trình. Tuy nhiên, từ trước đến nay, các tài liệu văn bản của các học giả, các nhà toán học ghi xuống dưới dạng chữ viết. Bạn X đã viết chương trình chuyển đổi các biểu thức này thành dạng số. Tuy nhiên chương trình của bạn còn chưa được hoàn thiện, *đặc biệt* là do các dấu ngoặc **đóng và mở** như "{", "}" , "(", ")" và "[", "]" . Việc mất cân bằng số lượng giữa các dấu này sẽ khiến máy tính không hiểu và không render được. Hãy giúp X viết chương trình chỉ ra đâu là biểu thức toán học bị sai.

<https://crohme2023.ltu-ai.dev/>

Lưu ý: Trong chuỗi có cả dấu khoảng trắng.

Input:

- Dòng đầu tiên chứa chuỗi s ($1 \leq N \leq 10^5$).

Output:

- Nếu hợp lệ hãy xuất ra **1**, còn không hợp lệ xuất ra **0**

Ví dụ:

Input	Output
$p_{\{\frac{5}{2}\}^{\{-\frac{1}{2}\}}_1$	1
$\int^4 p$	0

➤ Ý tưởng:

Duyệt qua chuỗi và đưa dấu "{", "(" và "[" vào 1 vector với biến đánh dấu vị trí sau cùng. Khi gặp dấu đóng ngoặc thì ta đối chiếu với biến đó nếu không thỏa thì in ra kết quả còn ngược lại thì tiếp tục cho đến hết chuỗi.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    string s;
    getline(cin, s);
    vector<char> v;
    for (auto &c : s)
    {
        if (c == '{' || c == '(' || c == '[')
            v.push_back(c);

        else if (c == '}' || c == ')' || c == ']')
        {
            if (v.empty())
            {
                cout << '0';
                return 0;
            }
            char top = v.back();
            v.pop_back();
            if ((c == '}' && top != '{') || (c == ')' && top != '(') || (c == ']' && top != '['))
            {
                cout << '0';
                return 0;
            }
        }
    }
    if (!v.empty())
        cout << '0';
    else
        cout << '1';

    return 0;
}
```

1.2. LinkedList-Insertion

➤ Đề bài:

Chèn một nút vào một danh sách liên kết đơn đã có thứ tự

Cho một danh sách liên kết đơn L gồm N số đã có thứ tự tăng dần và một số nguyên x . Bạn hãy chèn một node có giá trị x vào danh sách liên kết L trên sao cho danh sách liên kết vẫn bảo đảm có thứ tự tăng dần.

Input:

- Dòng đầu tiên số lượng phần tử của N ($0 \leq N \leq 10^5$).
- N dòng tiếp theo sẽ là giá trị của danh sách liên kết L (có thứ tự tăng dần).
- Dòng cuối cùng là giá trị x cần được chèn vào danh sách L nêu trên.

Output:

Danh sách liên kết L , mỗi số cách nhau bằng 1 dấu khoảng trắng.

Ví dụ:

Input	Output
5 1 3 4 6 7 5	1 3 4 5 6 7

➤ Ý tưởng:

Ở đây, ta chỉ cần viết hàm chèn nút vào danh sách là: **insertSortedLinkedList**

➔ Ta duyệt danh sách từ đầu tới cuối rồi so sánh với giá trị của nút đó để chèn vào.

➤ Script:

```
SinglyLinkedListNode* insertSortedLinkedList(SinglyLinkedListNode* head, int x) {
    SinglyLinkedListNode* newNode = new SinglyLinkedListNode(x);

    if(!head || head->data >= x){
        newNode->next = head;
        return newNode;
    }

    SinglyLinkedListNode* cur = head;
    while(cur->next && cur->next->data < x){
        cur = cur->next;
    }
    newNode->next = cur->next;
    cur->next = newNode;
    return head;
}
```

1.3. LinkedList-NhapDaThuc

➤ Đề bài:

Định nghĩa Đa thức một biến $f(x)$ có dạng: $S = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$

Viết chương trình nhập xuất đa thức một biến $f(x)$. Sau đó nhập giá trị biến x và tính giá trị $f(x)$ với kết quả hiển thị chính xác 2 số sau dấu thập phân.

INPUT

Dãy các số trong đó:

- Số nguyên đầu tiên ($0 \leq n \leq 100$): số lượng đơn thức trong đa thức
- n cặp số tiếp theo, mỗi cặp số là một đơn thức gồm: hệ số (số thực) và số mũ (số nguyên ≥ 0)
- Các đơn thức được nhập vào theo thứ tự giảm dần của số mũ, và không có đơn thức nào có cùng số mũ

OUTPUT

Như ví dụ bên dưới.

Lưu ý xuất đa thức:

- Biến trong đa thức ký hiệu là x
- Số mũ ký hiệu $^$
- Phép nhân không ghi ký hiệu
- Các ký tự biểu diễn đa thức ghi liền nhau (không khoảng trắng)
- Đơn thức đầu tiên nếu hệ số là số dương thì không được xuất dấu + trước hệ số
- Đơn thức có hệ số bằng 0 thì không xuất đơn thức đó
- Đơn thức có hệ số bằng 1 hoặc -1 thì không xuất số 1
- Đơn thức có số mũ bằng 0 thì chỉ xuất hệ số của đơn thức
- Đơn thức có số mũ bằng 1 thì không xuất số mũ

EXAMPLE

Input	Output
7 2 15 -1 10 0 8 1 5 6 2 -8.5 1 -10.52 0 2	Da thuc vua nhap la: $2x^{15} - x^{10} + x^5 + 6x^2 - 8.5x - 10.52$ Voi $x=2$, gia tri da thuc la: 64540.48
4 0 20 -10 6 1 5 -1 0 3.55	Da thuc vua nhap la: $-10x^6 + x^5 - 1$ Voi $x=3.55$, gia tri da thuc la: -19452.85
3 0 4 0 3 0 1 10	Da thuc vua nhap la: 0 Voi $x=10$, gia tri da thuc la: 0.00

➤ Ý tưởng:

Bài này thì ta chỉ cần viết hàm nhập xuất và tính giá trị $f(x)$ đúng theo yêu cầu là được.

➤ Script:

```
void Nhap(DATHUC &B, double heso, int somu){
    DONTHUC* newDonthuc = new DONTHUC(heso, somu);
    Node* newNode = new Node(newDonthuc);
    if(!B.head) B.head = B.tail = newNode;
    else {
        B.tail->next = newNode;
        B.tail = newNode;
    }
}
```

```

void Xuat(DATHUC B){
    if(!B.head)
        cout << 0;

    else{
        Node* cur = B.head;
        bool hasOutput = true;

        while(cur){
            double heso = cur->data->heso;
            int somu = cur->data->somu;

            if(heso == 0){
                cur = cur->next;
                continue;
            }
            if(!hasOutput && heso > 0) cout << "+";
            if(somu == 0) cout << heso;
            else{
                if(heso == -1) cout << "-";
                else if(heso != 1) cout << heso;
                cout << "x";
                if(somu != 1) cout << "^" << somu;
            }
            hasOutput = false;
            cur = cur->next;
        }
        if(hasOutput) cout << 0;
    }
}

double TinhDaThuc(DATHUC B, double x){
    if(!B.head)
        return 0;

    Node* cur = B.head;
    double sum = 0;
    while(cur){
        const double heso = cur->data->heso;
        int somu = cur->data->somu;
        sum += heso * pow(x, somu);
        cur = cur->next;
    }
    return sum;
}

```


1.4. LinkedList-Reverse

➤ Đề bài:

Cho danh sách liên kết chứa N phần tử. Bạn hãy đảo ngược danh sách liên kết này và in danh sách trên một dòng.

Input:

- Dòng đầu tiên chứa số nguyên N là số lượng phần tử trong danh sách liên kết.
- N dòng tiếp theo mỗi dòng chứa một số nguyên các phần tử theo thứ tự trong DSLK.

Output:

- Một dòng gồm N số nguyên là các phần tử sau khi đã được đảo ngược.

Ví dụ:

Input	Output
4	7 5 4 2
2	
4	
5	
7	

➤ Ý tưởng:

Bài này yêu cầu ta viết hàm nhập node, đảo ngược danh sách liên kết và in danh sách ra. Với hàm nhập node thì ta viết như bình thường, hàm đảo ngược thì ta duyệt từng cặp từ đầu tới cuối để thiết lập con trỏ next ngược lại theo chiều đảo và cuối cùng là hàm in.

➤ Script:

```
void insert_node(SinglyLinkedList* list,int node_data) {
    SinglyLinkedListNode* newNode = new SinglyLinkedListNode(node_data);
    if (!list->head) {
        list->head = newNode;
    } else {
        list->tail->next = newNode;
    }

    list->tail = newNode;
}

void reverseLinkedList(SinglyLinkedList* list){
    SinglyLinkedListNode* cur = list->head;
    SinglyLinkedListNode* next = nullptr;
    SinglyLinkedListNode* pre = nullptr;
    while(cur){
        next = cur->next;    // Lưu node kế tiếp
        cur->next = pre;     // Đảo ngược liên kết
        pre = cur;          // Dời pre và cur
        cur = next;         // tới trước
    }
    list->tail = list->head;
    list->head = pre;
}

void printLinkedList(SinglyLinkedList* list){
    SinglyLinkedListNode* cur = list->head;
    while(cur){
        cout<< cur->data << ' ';
        cur = cur->next;
    }
}
```

1.5. Dec to Bin

➤ Đề bài:

Chuyển đổi cơ số:

Việc chuyển đổi cơ số từ hệ thập phân sang hệ nhị phân là vô cùng quan trọng. Bởi vì hệ **nhị phân 0 và 1** là cách hiểu của mọi thiết bị máy tính hiện nay, cũng như các mạch logic (AND, OR, XOR, NOT, v.v.).

Bạn hãy viết chương trình giúp chuyển đổi từ hệ cơ số 10 sang hệ cơ số 2 (nhị phân)

Input:

- Dòng đầu tiên chứa số nguyên x ($1 \leq N \leq 10^5$), là số tại hệ cơ số 10.

Output:

- Giá trị nhị phân tương ứng với số x .

Ví dụ:

Input	Output
10	1010
1009	1111110001

➤ Ý tưởng:

Với cách thực hiện chuyển đổi như [sau](#), ta có thể viết được code tính toán cho bài này.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n;
    cin >> n;
    vector<int> v;
    while(n > 0){
        if(n % 2 == 1)
            v.push_back(1);

        if(n % 2 == 0)
            v.push_back(0);
        n /= 2;
    }
    for(int i = v.size() - 1; i >= 0; i--)
        cout << v[i];
    return 0;
}
```

1.6. LinkedList: MergetwoSortedLinkedList

➤ Đề bài:

Cho hai mảng đã có thứ tự, hãy trộn chúng thành một mảng cũng có thứ tự.

INPUT

Dòng đầu tiên chứa số T ($T < 10$) là số test case.

Mỗi test case có 3 hàng. Hàng đầu tiên chứa hai số n và m ($n, m < 500.000$) lần lượt là số phần tử của hai mảng A và B. Hàng thứ hai chứa n số nguyên được sắp theo thứ tự không giảm, đây chính là các phần tử của mảng A. Hàng cuối cùng chứa m số nguyên được sắp theo thứ tự không giảm, đây chính là các phần tử của mảng B.

OUTPUT

Với mỗi test case, xuất ra trên một dòng mảng có thứ tự thu được bằng cách trộn hai mảng A và B

VÍ DỤ

INPUT	OUTPUT
1 5 4 1 2 5 7 10 2 4 9 11	1 2 2 4 5 7 9 10 11
2 95 21 2 3 3 3 3 3 5 6 7 8 9 10 12 14 15 17 19 21 22 23 23 24 24 26 28 29 30 30 31 33 33 35 37 38 38 39 39 39 39 39 39 40 40 40 42 43 45 46 48 48 50 51 52 54 55 57 59 59 60 60 61 63 64 64 66 68 70 72 72 74 75 75 75 76 76 76 77 79 81 82 83 83 83 83 84 86 88 90 91 93 94 0 0 2 2 3 4 4 5 7 7 9 9 11 11 11 13 14 16 17 19 20 65 26 2 3 5 6 7 9 9 9 11 13 14 14 15 16 17 19 19 20 21 21 22 22 24 26 28 28 29 29 29 30 30 32 32 32 34 35 36 38 39 40 41 42 42 43 44 46 47 49 50 50 51 53 54 54 54 56 57 57 58 58 60 62 64 66 66 0 0 2 4 6 6 6 8 8 9 9 11 12 13 15 16 18 20 21 21 23 23 25 26 26 26	0 0 2 2 2 3 3 3 3 3 3 3 4 4 5 5 6 7 7 7 8 9 9 9 10 11 11 11 12 13 14 14 15 16 17 17 19 19 20 21 22 23 23 24 24 26 28 29 30 30 31 33 33 35 37 38 38 38 39 39 39 39 39 39 40 40 40 42 43 45 46 48 48 50 51 52 54 55 57 59 59 60 60 61 63 64 64 66 68 70 72 72 74 75 75 75 76 76 76 77 79 81 82 83 83 83 83 84 86 88 90 91 93 94 0 0 2 2 3 4 5 6 6 6 7 8 8 9 9 9 9 11 11 12 13 13 14 14 15 15 16 16 17 18 19 19 20 20 21 21 21 21 22 22 23 23 24 25 26 26 26 26 28 28 29 29 29 30 30 32 32 32 34 35 36 38 39 40 41 42 42 43 44 46 47 49 50 50 51 53 54 54 54 56 57 57 58 58 60 62 64 66 66

➤ Ý tưởng:

Để trộn hai mảng, ta so sánh 2 nút đầu của từng mảng rồi dịch cái bé hơn về phía sau và tiếp tục như thế cho đến khi hết ít nhất 1 mảng thì ta chỉ cần nối đuôi của mảng mới này với đầu của mảng còn sót lại đó nữa là xong.

➤ Script:

```
SinglyLinkedListNode* mergeLists(SinglyLinkedListNode*
head_list1,SinglyLinkedListNode* head_list2){
    if(!head_list2) return head_list1;
    if(!head_list1) return head_list2;
    SinglyLinkedListNode* head = nullptr;
    SinglyLinkedListNode* tail = nullptr;
    if(head_list1->data < head_list2->data){
        head = tail = head_list1;
        head_list1 = head_list1->next;
    }
    else{
        head = tail = head_list2;
        head_list2 = head_list2->next;
    }
    while(head_list1 && head_list2){
        if(head_list1->data < head_list2->data){
            tail->next = head_list1;
            head_list1 = head_list1->next;
        }
        else{
            tail->next = head_list2;
            head_list2 = head_list2->next;
        }
        tail = tail->next;
    }
    if(head_list1) tail->next = head_list1;
    else tail->next = head_list2;
    return head;
}
```

1.7. Tree: Preorder Traversal(NLR) - Duyệt cây BST theo NLR

➤ Đề bài:

Tree: Preorder Traversal (Duyệt cây theo thứ tự NLR)

Implement the **function void preOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the NLR order traversal as a single line of the value of tree with a space separated.

Sinh viên cài đặt hàm **void preOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt NLR

Sample (Ví dụ):

Input	Output
6	1 2 5 3 4 6
1 2 5 3 4 6	

1	
\	
2	
\	
5	
/ \	
3 6	
\	
4	

➤ Ý tưởng:

Ở bài này, ta sử dụng phép đệ quy duyệt cây từ trái sang để nó theo thứ tự NLR.

➤ Script:

```
void preOrder(Node *root) {  
    if (root == nullptr) return;  
    cout << root->data << " ";  
    preOrder(root->left);  
    preOrder(root->right);  
}
```

1.8. Tree: Preorder Traversal (NLR) II - Duyệt cây BST theo NLR không đệ quy

➤ Đề bài:

Tree: Preorder Traversal II (Duyệt cây theo thứ tự NLR II)

Implement the **function void preOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the NLR order traversal as a single line of the value of tree with a space separated. Don't use recursion in this problem.

Sinh viên cài đặt hàm **void preOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt NLR. Sinh viên chú ý: không được sử dụng đệ quy.

Sample (Ví dụ):

Input	Output
6	1 2 5 3 4 6
1 2 5 3 4 6	

1	
\	
2	
\	
5	
/ \	
3 6	
\	
4	

➤ Ý tưởng:

Không được sử dụng đệ quy thì ta sẽ dùng stack vì nó là LIFO nên đẩy nút phải trước, nút trái sau để nút trái được xử lý trước và duyệt cây theo thứ tự NLR.

➤ Script:

```
void preOrder(Node *root) {
    if (root == nullptr) return;
    stack<Node*> stk;
    stk.push(root);
    while(!stk.empty()){
        Node* temp = stk.top();
        cout << temp->data << " ";
        stk.pop();
        if(temp->right) stk.push(temp->right);
        if(temp->left) stk.push(temp->left);
    }
}
```

1.9. LinkedList: Tìm nút chung của 2 danh sách liên kết đơn

➤ Đề bài:

Tìm nút (node) chung của 2 danh sách liên kết đơn Cho 2 danh sách liên kết đơn, hãy cài đặt hàm `SinglyLinkedListNode* findMergeNode(SinglyLinkedListNode* head1, SinglyLinkedListNode* head2)` để tìm nút chung (nếu có) của 2 danh sách liên kết đơn này.

Chú ý hàm `findMergeNode` có tham số `head1`, `head2` là 2 con trỏ trỏ đến nút đầu của 2 danh sách liên kết. Hàm này không nằm trong lớp nào cả. Hàm này trả về con trỏ trỏ đến nút chung của 2 danh sách liên kết. Nếu 2 danh sách liên kết không có nút chung thì hàm sẽ trả về giá trị `nullptr`.

```
class SinglyLinkedListNode {
public:
    int data;
    SinglyLinkedListNode *next;
    SinglyLinkedListNode(int node_data) {
        this->data = node_data;
        this->next = nullptr;
    }
};

class SinglyLinkedList {
public:
    SinglyLinkedListNode *head;
}
};
```

➤ Ý tưởng:

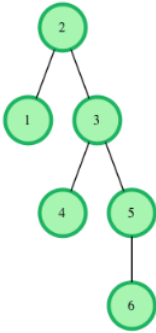
Ta sẽ so hai danh sách này để đưa chúng về cùng độ dài rồi duyệt từ đầu đến cuối của cặp vị trí tương ứng để tìm ra nút chung nếu không được thì trả về giá trị **nullptr**.

➤ Script:

```
SinglyLinkedListNode* findMergeNode(SinglyLinkedListNode* head1,
SinglyLinkedListNode* head2) {
    int len1 = 0, len2 = 0;
    SinglyLinkedListNode* temp1 = head1;
    SinglyLinkedListNode* temp2 = head2;
    while(temp1) {
        len1++;
        temp1 = temp1->next;
    }
    while(temp2) {
        len2++;
        temp2 = temp2->next;
    }
    temp1 = head1;
    temp2 = head2;
    if (len1 > len2) {
        for (int i = 1; i <= len1 - len2; i++) {
            temp1 = temp1->next;
        }
    }
    else if (len2 > len1) {
        for (int i = 1; i <= len2 - len1; i++) {
            temp2 = temp2->next;
        }
    }
    while (temp1 && temp2) {
        if (temp1 == temp2) {
            return temp1;
        }
        temp1 = temp1->next;
        temp2 = temp2->next;
    }
    return nullptr;
}
```

1.10. Binary Search Tree: Nút tổ tiên thấp nhất (tiếng Việt)

➤ Đề bài:



Cho một cây nhị phân tìm kiếm (BST), sinh viên hãy cài đặt hàm *lca* để tìm ra nút tổ tiên thấp nhất của cây BST đã cho với các tham số.

- root: con trỏ trỏ đến nút gốc

- v1, v2: giá trị 2 nút cần tìm nút tổ tiên gần nhất

Hàm này sẽ trả về con trỏ trỏ đến nút tổ tiên gần nhất của 2 nút

Ví dụ

Input

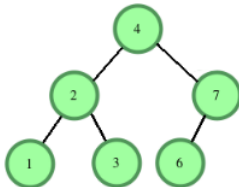
```
6
4 2 3 1 7 6
1 7
```

Output

Con trỏ trỏ đến nút có giá trị 4.

Diễn giải:

Sinh viên xem cây BST được tạo ra từ dữ liệu input ở trên



Với v1= 1 and v2= 7 thì hàm sẽ trả về con trỏ trỏ đến nút có giá trị là 4

➤ Ý tưởng:

Dễ thấy nếu cho hai nút có giá trị v1 và v2 thì nút chung của hai nút này là giá trị s sao cho: $\min(v1, v2) < s < \max(v1, v2)$ và nút đó phải nằm gần root nhất.

➤ Script:

```
Node *lca(Node *root, int v1, int v2) {
    if(root == nullptr || root->data == v1 || root->data == v2) return root;
    if(max(v1, v2) < root->data) return lca(root->left, v1, v2);
    else if(min(v1, v2) > root->data) return lca(root->right, v1, v2);
    return root;
}
```

2. Advance

2.1. Kiểm Kê 2

➤ Đề bài:

Cửa hàng X có nhiều loại hàng, mỗi loại hàng đều có mã khác nhau. Tuy nhiên, trong lúc bán hàng, nhân viên quên không nhập số lượng mỗi loại mà chỉ nhập mã các loại hàng trên từng dòng. Bạn hãy cho biết với mỗi loại hàng thì cửa hàng đã bán được bao nhiêu?

Ví dụ: Có một số loại hàng có mã lần lượt là [123, 45, 8, 19, 1, 2333].

Bạn nhân viên sẽ nhập theo từng dòng [123, 45, 45, 8, 1, 123, 123, 8, 8].

Từ đây có thể biết:

- Với loại hàng có mã là 1 thì cửa hàng đã bán 1.
- Với loại hàng có mã là 8 thì cửa hàng đã bán 3.
- Với loại hàng có mã là 45 thì cửa hàng đã bán 2.
- Với loại hàng có mã là 123 thì cửa hàng đã bán 3.

Input:

- Dòng đầu tiên là số nguyên ($1 \leq N \leq 5 \times 10^4$) là số lượng mã loại hàng nhập.
- N dòng tiếp theo, mỗi dòng là một mã của loại hàng được nhập, độ dài của mã có thể lên tới 100.

Output:

- Gồm nhiều dòng, mỗi dòng gồm 2 thành phần là mã số của một loại hàng và số lượng loại hàng đó. In kết quả theo thứ tự **giảm dần theo số lượng từng loại hàng**, nếu **số lượng bằng nhau** thì in theo **mã số tăng dần**.

Chú ý:

- So sánh 2 mã theo thứ tự số tự nhiên.
- Sử dụng '\n' để xuống dòng thay vì endl để tránh bị **Time Limit Exceeded (TLE)**

➤ Ý tưởng:

Sắp xếp các loại hàng nhập vào theo thứ tự, sau đó đưa các cặp mã và số lượng hàng của mã đó vào 1 vector. Sắp xếp vecto này theo thứ tự **giảm dần theo số lượng từng loại hàng**, nếu **số lượng bằng nhau** thì in theo **mã số tăng dần**.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<string> &arr, int low, int high)
{
    string pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true)
    {
        do
        {
            i++;
        } while (arr[i] < pivot);

        do
        {
            j--;
        } while (arr[j] > pivot);

        if (i >= j)
            return j;

        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<string> &arr, int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        QuickSort(arr, low, pi);
        QuickSort(arr, pi + 1, high);
    }
}
```

```

int partition(vector<pair<string, int>> &arr, int low, int high, bool
(*comp)(pair<string, int>, pair<string, int>))
{
    pair<string, int> pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true)
    {
        do
        {
            i++;
        } while (comp(arr[i], pivot));
        do
        {
            j--;
        } while (comp(pivot, arr[j]));

        if (i >= j)
            return j;

        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<pair<string, int>> &arr, int low, int high, bool
(*comp)(pair<string, int>, pair<string, int>))
{
    if (low < high)
    {
        int pi = partition(arr, low, high, comp);
        QuickSort(arr, low, pi, comp);
        QuickSort(arr, pi + 1, high, comp);
    }
}

bool comp(pair<string, int> a, pair<string, int> b)
{
    if (a.se != b.se)
        return a.se > b.se;

    string sa = a.fi, sb = b.fi;
    sa.erase(0, sa.find_first_not_of('0'));
    sb.erase(0, sb.find_first_not_of('0'));
    if (sa.empty())
        sa = "0";

```

```

    if (sb.empty())
        sb = "0";
    if (sa.size() != sb.size())
        return sa.size() < sb.size();
    return sa < sb;
}

int main()
{
    int t;
    cin >> t;
    vector<string> a(t);
    vector<pair<string, int>> b;

    for (int i = 0; i < t; i++)
    {
        cin >> a[i];
    }
    QuickSort(a, 0, t - 1);
    int c = 1;
    for (int i = 0; i < t - 1; i++)
    {
        if (a[i] != a[i + 1])
        {
            b.push_back({a[i], c});
            c = 1;
            continue;
        }
        else
        {
            c++;
        }
    }
    b.push_back({a[t - 1], c});
    QuickSort(b, 0, b.size() - 1, comp);
    for (auto &p : b)
    {
        cout << p.fi << " " << p.se << "\n";
    }
    return 0;
}

```

2.2. Binary Search 2

➤ Đề bài:

Binary Search 2

Cho một dãy số nguyên A gồm N phần tử $A_1, A_2, \dots, A_N$.

Có Q truy vấn, mỗi truy vấn có dạng **type x y**.

- Nếu **x = 1**, thì phải in ra vị trí đầu tiên xuất hiện **y** trong dãy A. Nếu không có thì in ra -1.
- Nếu **x = 2**, thì phải in ra vị trí cuối cùng xuất hiện **y** trong dãy A. Nếu không có thì in ra -1

Input:

- Dòng đầu tiên là 2 số nguyên dương N, Q ($1 \leq N \leq 10^5, 1 \leq Q \leq 5 \times 10^5$)
- Dòng tiếp theo chứa N số nguyên A_i ($-10^9 \leq A_i \leq 10^9$) là các phần tử của mảng.
- Q dòng tiếp theo, mỗi dòng có dạng **type x y**.

Output:

Gồm Q dòng, mỗi dòng gồm một câu trả lời ứng với từng truy vấn.

Ví dụ:

Input	Output
13 6	-1
9 10 12 7 -1 1 1 0 1 7 9 9 1	-1
type 1 3	6
type 2 2	13
type 1 1	1
type 2 1	10
type 1 9	
type 2 7	

➤ Ý tưởng:

Ta tìm vị trí xuất hiện đầu tiên và cuối cùng của y trước, nếu không có thì in ra -1, ngược lại tùy vào x mà ta in ra giá trị tương ứng phù hợp. Để tìm vị trí (iterator) này, tôi sử dụng hàm **lower_bound** và **upper_bound** với độ phức tạp $O(\log n)$, n là độ dài mảng sẽ cho ra kết quả nhanh và chính xác nhất.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second

int partition(vector<pair<int, int>> &arr, int low, int high, bool
(*comp)(pair<int, int>, pair<int, int>))
{
    pair<int, int> pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true)
    {
        do
        {
            i++;
        } while (comp(arr[i], pivot));

        do
        {
            j--;
        } while (comp(pivot, arr[j]));

        if (i >= j)
            return j;

        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<pair<int, int>> &arr, int low, int high, bool
(*comp)(pair<int, int>, pair<int, int>))
{
    if (low < high)
    {
        int pi = partition(arr, low, high, comp);

        QuickSort(arr, low, pi, comp);
        QuickSort(arr, pi + 1, high, comp);
    }
}
```



```

bool comp(pair<int, int> a, pair<int, int> b)
{
    if (a.fi != b.fi)
        return a.fi < b.fi;
    return a.se < b.se;
}

int main()
{
    int N, Q;
    cin >> N >> Q;
    vector<pair<int, int>> v;

    for(int i = 1; i <= N; i++){
        int temp;
        cin >> temp;
        v.push_back({temp, i});
    }

    QuickSort(v, 0, v.size() - 1, comp);

    for (int i = 0; i < Q; ++i) {
        int x, y;
        string type;
        cin >> type >> x >> y;

        auto lower_it = lower_bound(v.begin(), v.end(), make_pair(y, 0));
        auto upper_it = upper_bound(v.begin(), v.end(), make_pair(y, N + 1));

        if (lower_it == v.end() || lower_it->first != y)
            cout << -1 << "\n";
        else {
            if (x == 1)
                cout << lower_it->second << "\n";
            else if (x == 2)
                cout << (upper_it - 1)->second << "\n";
        }
    }

    return 0;
}

```

2.3. khangtd.DetectVirusin2D

➤ Đề bài:

Bây giờ, file dữ liệu được tổ chức dưới dạng một bảng hình chữ nhật gồm $N \times M$ ô, mỗi ô chứa một ký tự thường từ **a..z**. Sau đó sẽ có một tập danh sách gồm Q mẫu virus, mỗi mẫu có độ dài từ 2 đến 10. Nhiệm vụ của bạn là với mỗi mẫu virus trong danh sách, hãy xác định xem mẫu virus có nằm trong file đó hay không? (mẫu virus có thể khớp theo chiều ngang hoặc theo chiều dọc)

Dữ Liệu:

Dòng đầu tiên gồm 3 số nguyên dương N , M , Q thể hiện kích thước của file và số lượng mẫu virus trong danh sách

N dòng tiếp theo, mỗi dòng gồm M ký tự biểu diễn bảng chữ cái

Q dòng tiếp theo, mỗi dòng gồm một mẫu virus độ dài từ 2 đến 10 ký tự.

Kết quả:

gồm một dòng duy nhất gồm Q ký tự. Ký tự thứ i là '1' nếu như file bị nhiễm mẫu virus thứ i , ngược lại là '0'.

Điều kiện:

$$1 \leq N \cdot M \leq 10^5$$

$$1 \leq Q \leq 10^4$$

Ví dụ:

INPUT		OUTPUT
4 5 4 attnt oboot campc ontes tt attn xyc aoco		1101

➤ Ý tưởng:

Sử dụng thuật toán [Kabin-Karp](#) để so khớp chuỗi bởi để tính hash của 1 chuỗi con trong hàng hoặc cột chỉ tốn $O(1)$ và đem so với hash của các mẫu sẽ cho ra kết quả nhanh hơn rất nhiều.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
using ull = unsigned long long;
// Cơ sở dùng cho Kabin-Karp
const ull BASE = 131;

// Hàm tính prefix hash cho một chuỗi
vector<ull> computePrefixHash(const string &s) {
    int n = s.size();
    vector<ull> prefix(n + 1, 0);
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] * BASE + s[i];
    }
    return prefix;
}

// Hàm tính lũy thừa cho một độ dài nhất định
vector<ull> computePow(int n) {
    vector<ull> pow(n + 1, 1);
    for (int i = 0; i < n; i++) {
        pow[i + 1] = pow[i] * BASE;
    }
    return pow;
}

// Hàm lấy hash của substring s[l, r-1] sử dụng prefix hash đã tính trước
ull getSubHash(const vector<ull>& prefix, const vector<ull>& pow, int l, int r) {
    return prefix[r] - prefix[l] * pow[r - l];
}

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    vector<string> Matrix(n);
    for (int i = 0; i < n; i++) {
        cin >> Matrix[i];
    }

    vector<string> patt(q);
    int max_len = 0;
    for (int i = 0; i < q; i++) {
        cin >> patt[i];
        max_len = max(max_len, (int)patt[i].size());
    }
}
```

```

    // Chuẩn bị mảng lữ thừa với độ dài tối đa là  $\max(m, n) + 1$ , vì chuỗi hàng
    có độ dài  $m$ , cột có độ dài  $n$ 
    int maxSize = max(m, n);
    vector<ull> pow = computePow(maxSize);

    // Sử dụng vector của unordered_set để lưu các hash theo độ dài
    // lst_hash[len] sẽ chứa tất cả hash của các chuỗi con có độ dài = len
    vector<unordered_set<ull>> lst_hash(max_len + 1);

    // Xét từng hàng:
    for (int i = 0; i < n; i++) {
        const string &row = Matrix[i];
        int lenRow = row.size();
        // Tính prefix hash cho hàng
        vector<ull> prefix = computePrefixHash(row);
        for (int length = 1; length <= max_len && length <= lenRow; length++) {
            for (int j = 0; j <= lenRow - length; j++) {
                ull h = getSubHash(prefix, pow, j, j + length);
                lst_hash[length].insert(h);
            }
        }
    }

    // Xét từng cột:
    for (int col = 0; col < m; col++) {
        string colStr;
        colStr.reserve(n);
        for (int row = 0; row < n; row++) {
            colStr.push_back(Matrix[row][col]);
        }
        int lenCol = colStr.size();
        // Tính prefix hash cho cột
        vector<ull> prefix = computePrefixHash(colStr);
        for (int length = 1; length <= max_len && length <= lenCol; length++) {
            for (int j = 0; j <= lenCol - length; j++) {
                ull h = getSubHash(prefix, pow, j, j + length);
                lst_hash[length].insert(h);
            }
        }
    }
}

```

```

// Với mỗi mẫu cần tìm, tính hash và kiểm tra trong tập hợp
for (int i = 0; i < q; i++) {
    const string &pattern = patt[i];
    // Tính hash của mẫu
    int lenPattern = pattern.size();
    ull hashPat = 0;
    for (char c : pattern) {
        hashPat = hashPat * BASE + c;
    }
    // Nếu tìm thấy, in '1', không tìm thấy in '0'
    cout << (lst_hash[lenPattern].count(hashPat) ? '1' : '0');
}

return 0;
}

```

2.4. khangtd.XepHang2

➤ Đề bài:

Một lớp có n sinh viên. Các sinh viên có mã số sinh viên (mssv) theo thứ tự từ 1 đến n . Ban đầu các bạn sinh viên xếp hàng theo đúng thứ tự theo mã số sinh viên từ 1 đến n , nghĩa là sinh viên có mssv là 1 ở đầu, tiếp theo là sinh viên có mssv là 2,..., sinh viên có mssv là n ở đứng cuối cùng. Khi thầy giáo gọi một bạn nào đó, thì bạn đó lên đứng đầu hàng. Cứ mỗi lần gọi trong m lần gọi thì thầy giáo muốn biết sinh viên đứng cuối cùng là bạn nào?



Dữ liệu:

- Dòng thứ nhất gồm hai số nguyên n, m cách nhau một khoảng trắng ($1 \leq n, m \leq 10^5$). n là số sinh viên và m là số lần thầy giáo gọi.
- Dòng tiếp theo gồm m số nguyên $a_1, a_2, \dots, a_m$ mỗi số cách nhau một khoảng trắng ($1 \leq a_i \leq n$). a_i là mssv của sinh viên được gọi trong lần gọi thứ i .

Kết quả:

Là m số nguyên, mỗi số cách nhau một khoảng trắng, thể hiện là mssv của học sinh cuối cùng sau khi thầy gọi.

Ví dụ:

INPUT		OUTPUT
5 2		4 3
5 4		

➤ Ý tưởng:

Bài này đơn giản là ta chỉ cần gọi 1 cái `unordered_map` tới con trỏ của giá trị trong danh sách mssv. Khi thầy gọi tới mssv của học sinh nào thì ta sẽ truy xuất xem còn con trỏ của giá trị đó trong danh sách không, nếu còn thì xóa nó ở danh sách chính rồi đưa lên đầu hàng và cập nhật con trỏ của nó. Cuối cùng là in ra mssv còn ở cuối danh sách là xong.

Cách này giúp giảm độ phức tạp của việc duyệt lại danh sách mssv và cũng như làm cho đoạn code gọn gàng và chính xác hơn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    list<int> call;
    unordered_map<int, list<int>::iterator> pos;

    // Lập danh sách từ 1 tới n và trỏ vị trí
    for (int i = 1; i <= n; ++i) {
        call.push_back(i);
        pos[i] = prev(call.end());
    }

    // Đặt lại vị trí cùng con trỏ nó sau đó in ra phần tử cuối
    for (int i = 0; i < m; ++i) {
        int x;
        cin >> x;

        if (pos.count(x)) {
            call.erase(pos[x]);
        }
        call.push_front(x);
        pos[x] = call.begin();
        cout << call.back() << " ";
    }

    return 0;
}
```


2.5. khangtd.XepHang

➤ Đề bài:

Một lớp có n sinh viên. Các sinh viên có mã số sinh viên (mssv) theo thứ tự từ 1 đến n . Ban đầu các bạn sinh viên xếp hàng theo đúng thứ tự theo mã số sinh viên từ 1 đến n , nghĩa là sinh viên có mssv là 1 ở đầu, tiếp theo là sinh viên có mssv là 2,..., sinh viên có mssv là n ở đứng cuối cùng. Khi thầy giáo gọi một bạn nào đó, thì bạn đó lên đứng đầu hàng. Hỏi sau m lần được gọi, thì thứ tự của các bạn sinh viên sẽ như thế nào?



Dữ liệu:

- Dòng thứ nhất gồm hai số nguyên n, m cách nhau một khoảng trắng ($1 \leq n, m \leq 10^5$). n là số sinh viên và m là số lần thầy giáo gọi.
- Dòng tiếp theo gồm m số nguyên $a_1, a_2, \dots, a_m$ mỗi số cách nhau một khoảng trắng ($1 \leq a_i \leq n$). a_i là mssv của sinh viên được gọi trong lần gọi thứ i .

Kết quả:

Là m số nguyên, mỗi số cách nhau một khoảng trắng, thể hiện là mssv từ đầu hàng đến cuối hàng.

Ví dụ:

INPUT		OUTPUT
5 2		4 5 1 2 3
5 4		

➤ Ý tưởng:

Bài này cũng gần giống ở bài trên ở đoạn phải cập nhật lại con trỏ sau mỗi lần thầy gọi tới. Ở đây, ta sẽ không khai báo danh sách mssv của tất cả sinh viên ngay từ đầu để khi thao tác sẽ công kênh mà thay vào đó sẽ lập thành 2 danh sách. 1 là `list<int>` **call** khi được gọi sẽ đưa lên đầu và xóa đi nó trước đó (nếu có). 2 là `unordered_set<int>` **seen** để xác định xem mssv nào đã được gọi mục đích là để cuối cùng sẽ gọi lại tất cả các mssv theo thứ tự nếu chưa được gọi.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    list<int> call;
    unordered_map<int, list<int>::iterator> pos;
    unordered_set<int> seen;

    for (int i = 0; i < m; i++) {
        int temp;
        cin >> temp;

        if (pos.count(temp)) {
            call.erase(pos[temp]);
        }
        call.push_front(temp);
        pos[temp] = call.begin();
        seen.insert(temp);
    }

    for (int x : call)
        cout << x << " ";

    for (int i = 1; i <= n; i++) {
        if (seen.find(i) == seen.end()) {
            cout << i << " ";
        }
    }

    return 0;
}
```